

UniMat AI - Complete Project & Codebase Architecture Guide

1. Introduction: What Problem Are We Solving?

In large industrial organizations (like oil refineries, power plants, and manufacturing firms), different departments often purchase the exact same physical item but record it in their databases with completely different names, abbreviations, and formatting.

For example, one engineer might log a purchase as `VLV-BALL-SS316-2IN-150#`, while another logs it as `Ball Valve 50mm SS316 150 Class`.

When this happens, the company's inventory system thinks these are two entirely different items. This leads to massive inefficiencies: overstocking, inaccurate inventory counts, and wasted procurement budgets.

UniMat AI is an intelligent pipeline designed to ingest this messy "Material Master Data," automatically understand the true semantic meaning behind the messy text, and group identical items together into unified "Clusters."

2. Core Technologies Used (Explained Simply)

Our project leverages a modern, cutting-edge AI stack. Here is exactly what we used and why:

FastAPI (The Backend Web Framework)

- **What it is:** A high-performance Python framework for building web APIs.
- **Analogy:** Think of FastAPI as the front desk receptionist at a hotel. When the frontend (the user interface) makes a request (like "Upload this Excel file"), FastAPI takes the request, routes it to the correct "department" (the AI pipeline), and brings the result back to the user.
- **Why we used it:** It is incredibly fast and supports "asynchronous" programming, meaning it can handle multiple file uploads at the same time without freezing.

PostgreSQL (The Relational Database)

- **What it is:** An enterprise-grade, highly reliable SQL database.
- **Analogy:** This is the company's giant, highly organized Excel spreadsheet. It stores all the raw text of the materials, the IDs, and keeps track of which items belong to which AI-generated cluster.
- **Why we used it:** Unlike simpler databases (like SQLite), PostgreSQL can handle massive datasets (hundreds of thousands of rows) securely and efficiently without corrupting data.

Qdrant (The Vector Database)

- **What it is:** A specialized database designed specifically for Artificial Intelligence. Instead of searching by exact text matches, it searches by "meaning" using spatial math.
- **Analogy:** Imagine a massive 3D galaxy where every star is a material item. Qdrant places items with similar meanings physically close together in this galaxy. So, "Ball Valve" and "VLV-BALL" are placed right next to each other, while "Pump" is placed millions of lightyears away.
- **Why we used it:** It allows us to instantly find identical materials

even if they share zero identical words.

Regex (Regular Expressions) & NLP (Natural Language Processing)

- **What it is:** A combination of pattern-matching code and a custom industrial dictionary.
- **Analogy:** It acts like a highly trained human proofreader. It reads the raw messy text, spots industrial abbreviations (like CS WCB or MOC), and expands them into their full English words (CARBON STEEL WCB, MATERIAL OF CONSTRUCTION). It also standardizes units (turning 2 IN and 2" both into 2 INCH).

SentenceTransformers (BGE-Base-EN-v1.5)

- **What it is:** An advanced Machine Learning model running entirely locally on our backend.
- **Analogy:** The translator. It reads the cleaned English text and translates it into a language the Qdrant database understands: a mathematical vector made of 768 decimal numbers.
- **Why we used it:** By running this model locally, we avoid expensive cloud computing costs and ensure 100% data privacy for the company.

Google Gemini (The Large Language Model)

- **What it is:** A generative AI model similar to ChatGPT.
 - **Analogy:** The final boss. After the other systems have grouped identical items into a cluster, Gemini looks at the entire group and uses its vast world knowledge to generate a perfect, standardized "Unified Name" and a hierarchical "Taxonomy Category" (e.g., Mechanical > Valves > Ball Valves).
-

3. The 6-Step Algorithmic Workflow

When a user uploads an Excel file, here is exactly what happens under the hood:

1. **Ingestion & Parsing (`app/routers/upload.py`):** The Excel file is parsed using the `pandas` library. The raw descriptions are saved into PostgreSQL.
 2. **NLP Preprocessing (`app/services/nlp_service.py`):** Every description is passed through our custom regex cleaner. Special characters are removed, dimensions are standardized, and hundreds of domain-specific abbreviations are expanded.
 3. **AI Vectorization (`app/services/vector_service.py`):** The cleaned text is passed into the `bge-base` embedding model. The model calculates the semantic context of the text and outputs a 768-dimensional vector coordinate.
 4. **Semantic Search (`app/services/pipeline.py`):** For each new vector, we query the Qdrant Vector Database: *"Find me any existing vectors that are >82% close to this new vector."*
 5. **Dynamic Clustering (`app/services/pipeline.py`):**
 - If a >82% match is found, the new item is flagged as a **Duplicate** and grouped into the existing item's cluster in PostgreSQL.
 - If no match is found, the new item is flagged as **Unique** and becomes the "parent" of a brand new cluster. Its vector is saved into Qdrant for future items to match against.
 6. **Generative Naming (`app/services/llm_service.py`):** Once all clustering is done, the system gathers all the items in a cluster and sends them to Google Gemini in batches. Gemini reads the messy variations and replies with a clean, standardized Unified Name and Category, which is saved back to PostgreSQL.
-

4. Real-World Trace: The Journey of 3 Materials

Let's trace exactly how our algorithm groups three different descriptions of the same valve.

The Input Data:

- **Item A:** VLV-BALL-SS316-2IN-150#-FLG
- **Item B:** Ball Valve 50mm SS316 150 Class
- **Item C:** 2 inch Stainless Steel Ball Valve 150 Flanged

Step 1: NLP Preprocessing

The `nlp_service.py` dictionary steps in. It expands VLV to VALVE, SS316 to STAINLESS STEEL 316, 150# to 150 CLASS, and FLG to FLANGED.

- **Item A Output:** VALVE BALL STAINLESS STEEL 316 2 INCH 150 CLASS FLANGED
- **Item B Output:** BALL VALVE 50 MM STAINLESS STEEL 316 150 CLASS
- **Item C Output:** 2 INCH STAINLESS STEEL BALL VALVE 150 CLASS FLANGED

Notice how much more similar they look now!

Step 2 & 3: Vectorization and Search

- **Item A** is processed first. Qdrant is empty, so it finds 0 matches. Item A becomes a **Unique** parent cluster. Its vector is saved to Qdrant.
- **Item B** is processed. It is converted to a vector. We query Qdrant. Even though "50 MM" and "2 INCH" are different text, the AI model knows they mean the same thing spatially. Qdrant reports a **94% similarity score** with Item A. Item B is marked as a **Duplicate** of Item A.
- **Item C** is processed. Qdrant reports a **98% similarity score** with Item A. Item C is marked as a **Duplicate** of Item A.

Step 4: LLM Naming

The pipeline sends this payload to Gemini: *"Generate a name for this cluster: [VLV-BALL-SS316-2IN-150#-FLG, Ball Valve 50mm SS316 150 Class, 2 inch Stainless Steel Ball Valve 150 Flanged]"*

Gemini responds with:

- **Taxonomy:** Valves & Fittings > Ball Valves
- **Unified Name:** 2" Stainless Steel 316 Ball Valve, 150 Class, Flanged Ends

Result: Three wildly different database entries have been perfectly deduplicated and standardized without any human intervention!

5. Codebase Walkthrough (File-by-File)

If you look into the `backend/` directory, here is what every file does in the workflow:

The Core Setup

- **`app/main.py`:** The entry point. It boots up the FastAPI server, connects to the database, and loads the API routes.
- **`app/config.py`:** Loads our passwords, API keys, and settings (like the PostgreSQL credentials and the `0.82` clustering threshold) from the `.env` file.
- **`app/database.py`:** Contains the connection logic for PostgreSQL using SQLAlchemy.

Data Structures

- **`app/models.py`:** Defines the physical SQL tables. It tells PostgreSQL to create a `materials` table (with columns for raw descriptions, cluster IDs, etc.) and a `sessions` table (to track upload progress).
- **`app/schemas.py`:** Uses Pydantic to validate data. If the frontend

tries to send a string when we expect an integer, `schemas.py` catches the error and blocks it.

API Endpoints (The Interfaces)

- **`app/routers/upload.py`**: Contains the code that accepts the physical `.xlsx` file upload from the frontend, uses `pandas` to extract the `Raw_Description` column, and saves the rows to PostgreSQL.
- **`app/routers/pipeline.py`**: Contains the endpoints the frontend calls to say *"Start processing session #123"* or *"Give me the progress percentage of session #123"*.

The AI Brains (The Services)

- **`app/services/nlp_service.py`**: The file containing our massive industrial abbreviation dictionary and Regex text-cleaning logic.
- **`app/services/vector_service.py`**: Handles downloading/loading the `SentenceTransformer` model locally. It also contains the code to connect to the Qdrant database and perform the semantic nearest-neighbor searches.
- **`app/services/llm_service.py`**: Contains the prompt engineering and API calls to Google Gemini to generate the final Taxonomy categories.
- **`app/services/pipeline.py`**: The master conductor. This script loops through the materials in the database, calls `nlp_service` to clean them, calls `vector_service` to embed and search them, updates their Cluster IDs in the database, and finally calls `llm_service` to name the clusters. It also calculates our final Accuracy Metrics.